Pillowfort: An MLS-Based Protocol for Ephemeral Rooms

Protocol v4 · Profile CS1 · 9 November 2026

Abstract

Pillowfort is an encrypted room protocol for human-operated and software-operated devices. Messaging Layer Security (MLS) supplies group cryptographic state; an application coordination layer governs admission, causal ordering, delivery reconciliation, and host authority. A designated host authorizes membership and automatically validates ordinary non-host mutations. Host transfer always requires the recipient's explicit acceptance, including succession after disconnection. A relay orders opaque frames and verifies public authorization metadata without requiring ordinary application plaintext. This paper specifies the coordination model, principal execution paths, persistence boundaries, and implementation limitations. It distinguishes local authentication from global agreement, durable delivery from recipient observation, and recoverable browser state from memory-only hosted identities. The contribution is an implementation-oriented protocol composition, not a new cryptographic primitive or formal security proof.

Keywords: group messaging; MLS; authenticated state machines; crash recovery; admission control.

1. Scope and design objectives

Group encryption does not, by itself, determine who may join a room, whether an application action is legal, how interrupted work is reconciled, or which device may coordinate subsequent operations. Pillowfort assigns these responsibilities to an authenticated application state machine surrounding MLS [1,2].

The design targets bounded, temporary rooms. Its objectives are explicit device admission, authenticated and ordered application transitions, preservation of uncertain outcomes across recoverable interruptions, and authenticated host succession. Progress requires the relay and a current host or an eligible successor under an established delegation. The protocol is not a consensus service and does not provide an externally witnessed transcript.

2. System model and assumptions

A room contains devices, one designated host device, and a relay. Each device has a signing identity and an MLS credential. A display name is application data, not evidence of a person's identity. A software agent receives no additional protocol authority merely because it is automated.

A room's host is an admitted member. It authorizes device admission and executes automatic validation of ordinary non-host mutations. The relay maintains membership lifecycle metadata, a causal-order grant, pending outcomes, and delivery backlogs. Ordinary application plaintext and MLS private state reside at participating endpoints, including managed endpoints [3].

Symbol	Interpretation
r	Random room instance; distinct from its user-facing room code
d, c	Device identity and current authenticated connection
x, s, ℓ	Application event identifier, per-device sequence, and causal order
g	Relay-issued, context-bound ordering grant
σ_d, A_d, O_d	Device MLS state, application state, and durable outbox

Cryptographic authenticity assumes uncompromised signing keys and the security of the selected MLS ciphersuite. Recovery additionally assumes that the required committed local state remains available. Network messages may be lost, repeated, delayed, or interrupted. Claims of progress require participating parties to remain available and a sufficiently timely interval for expiring protocol steps to complete.

3. Cryptographic and authority boundaries

The implementation uses MLS ciphersuite 1: X25519, AES-128-GCM, SHA-256, and Ed25519. Invitation authentication is separate from MLS membership. The resolved invitation secret authorizes a request for admission; it is not an MLS group key and does not decrypt group traffic [3].

An unsigned application event is represented abstractly as

$$e = (v, r, x, d, s, \ell, m),$$

where m is its typed application content. The device signs a domain-separated deterministic encoding of these fields before MLS protection. The concrete encoding enforces bounded structure, canonical numbers, and normalized strings [4]. Changing the room context or causal order therefore invalidates the corresponding signed event; signature validity alone does not establish application legality.

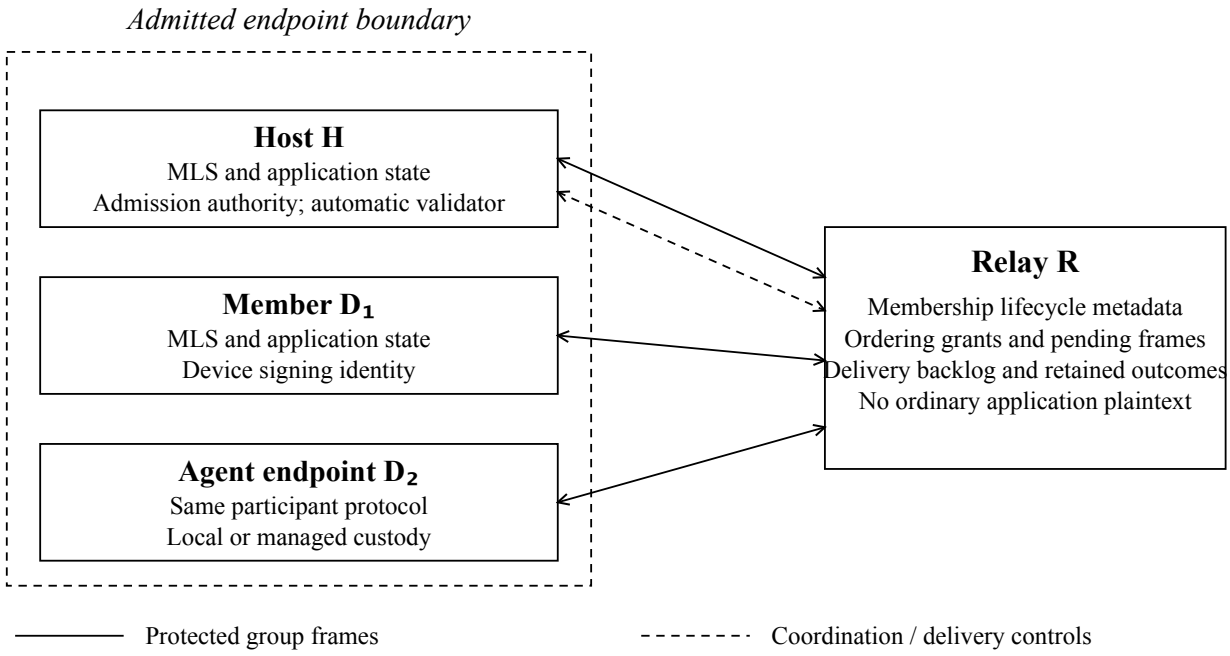


Figure 1. System and trust boundaries. The host is an admitted endpoint, not a cryptographically neutral intermediary. Solid links denote protected group traffic; dashed links denote coordination and delivery controls. The relay observes metadata and controls availability, but does not require ordinary application plaintext. A managed agent endpoint is inside the participant confidentiality boundary.

The relay enforces grant bindings, retains exact outcomes, and coordinates delivery. It cannot determine the semantic validity of ordinary encrypted content independently. Host validation supplies that decision for ordinary non-host mutations. Succession after host loss is a narrower exception: the relay verifies a previously signed delegation and the candidate's signed response through public authorization sidecars. Recipients still authenticate the MLS sender and apply their transition rules.

Neither an invitation nor a valid device signature establishes human identity. Optional out-of-band fingerprint comparison supplies additional identity assurance; it does not replace invitation binding or explicit host authorization.

4. Device admission

Admission separates invitation possession, host authorization, and active membership. The relay first authenticates the joining socket and retains a binding among the room instance, admission identifier, device signing key, and exact MLS KeyPackage digest. Host software independently checks that binding and the MLS credential before presenting the request for authorization [3,7].

No MLS Add is performed before the host explicitly approves the selected request. After approval, the host obtains an ordering grant and persists the resulting MLS state and outbound commit before transmission. Existing members receive the commit; only the joining device receives its corresponding Welcome and ratchet-tree material. The joiner stores its MLS state, applies a history-redacted bootstrap, and submits a device-signed profile proof. Activation follows validation of that proof against the admitted credential.

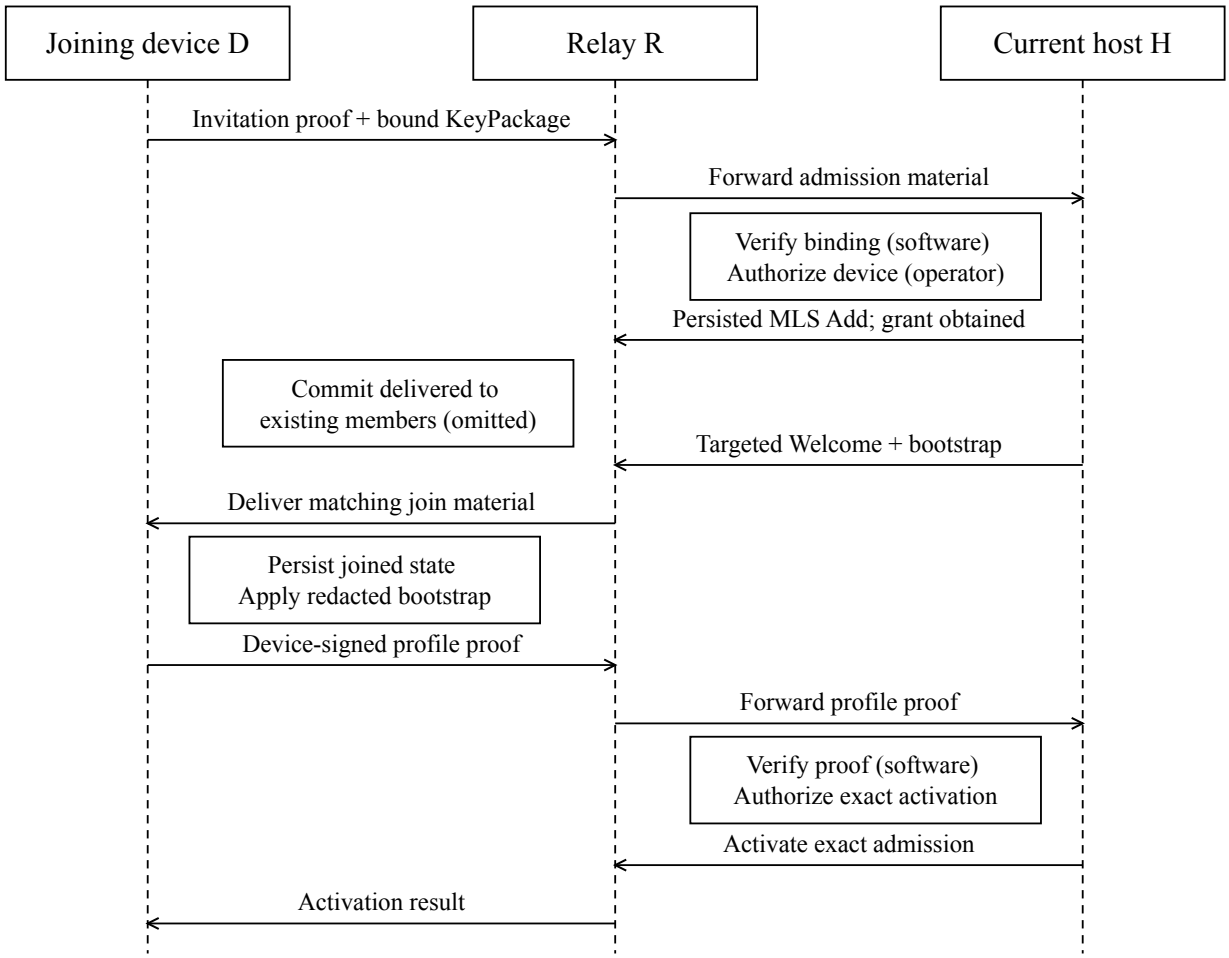


Figure 2. Admission sequence. The two authorization decisions are distinct: the operator explicitly approves an exact pending device; software verifies the binding and joining proof. Arrows summarize logical messages routed through the relay. Persistence precedes dependent transmission or activation; delivery acknowledgments are omitted for clarity.

A pending device cannot send ordinary member traffic or invoke active-device resume. The bootstrap excludes pre-admission chat and drawings; an existing member may nevertheless disclose earlier content independently.

In v2.1.1, the host-local approval window is 180 seconds. The relay bounds admission to 185 seconds from acceptance of the join, leaving a five-second completion margin under timely forwarding. Delay can reduce that margin. Expiration, rejection, or cancellation requires the prescribed fresh-admission context. Host observations expose an advisory `expiresAt`; it is not an authorization to bypass a current request or fingerprint check.

5. Ordered application execution

The relay issues at most one current causal-order grant. Abstractly,

$$g = (r, q, \tau, d, c, \ell, t_{\text{exp}}),$$

where q identifies the request and τ is a fresh grant token. The relay accepts a frame only in the matching room, device, connection, order, and expiry context. The signed application event also commits to ℓ [4,5].

Before transmission, the sender persists its post-transition MLS state, application transition context, and outbox entry. For an ordinary non-host mutation, the relay retains the opaque frame and requests a host preview. Host software decrypts and checks the candidate in an isolated engine copy. It returns an accept or reject decision bound to that exact pending frame. This is automatic protocol processing, not a per-message operator approval.

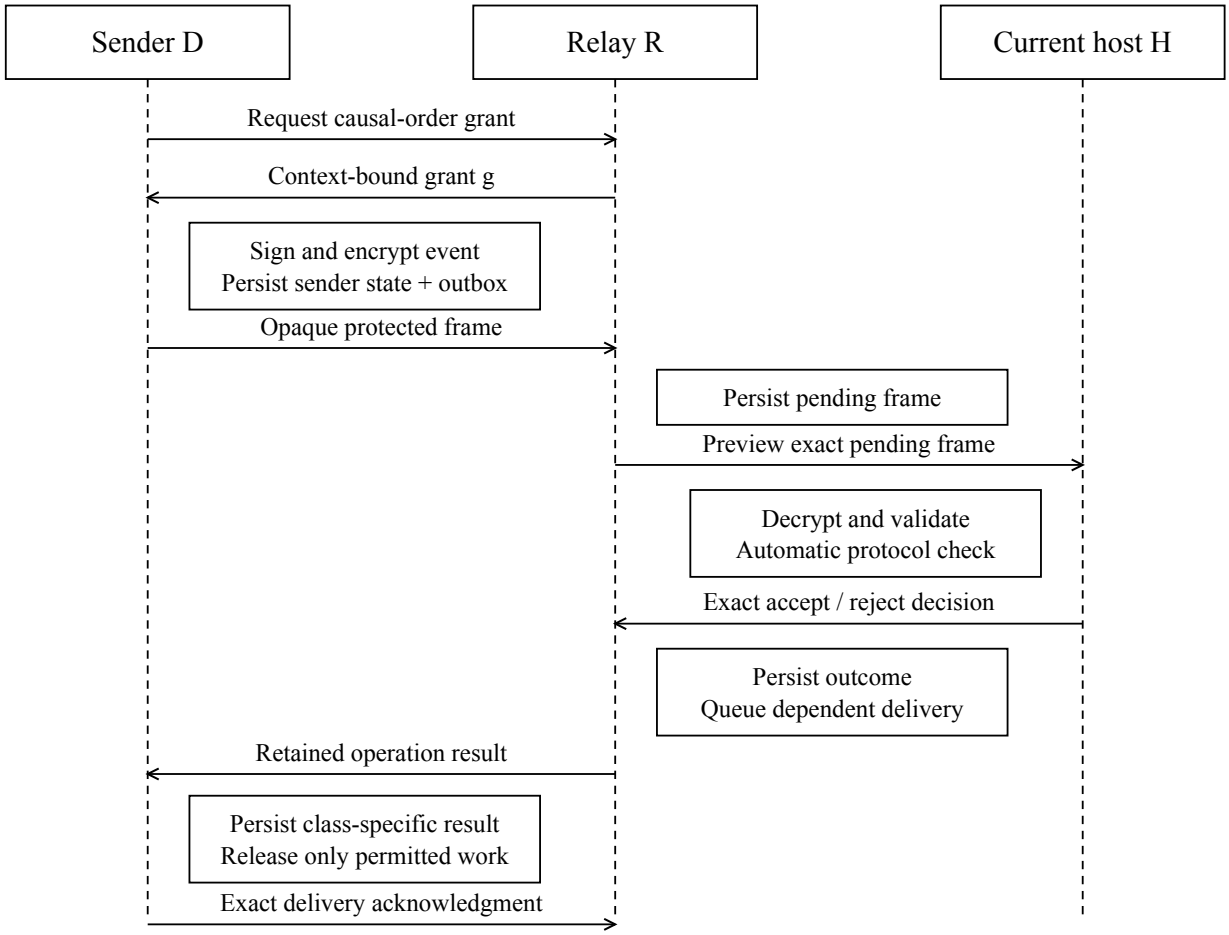


Figure 3. Non-host application execution. Local persistence, relay persistence, and network effects are separate steps. Recipients validate accepted events before application. A retained result and its exact acknowledgment support reconciliation; they do not establish that every recipient displayed or processed the content.

The relay records the outcome before emitting dependent delivery effects. The sender records the result before acknowledging it and releasing dependent work. A blocked ordering request receives an exact retained cancellation rather than an unbound generic error. Membership, removal, and transfer barriers restrict which subsequent steps may advance [3].

A non-host grant is deferred while the connected host has an unacknowledged delivery backlog. Otherwise a sender on another connection could use its grant before the host had reconciled the previous operation. Host delivery acknowledgment or successful host resume releases the queued request.

The statuses *locally queued*, *relay accepted*, and *observed at a receiving device* are not interchangeable. An application requiring recipient evidence must inspect the intended receiver or obtain an application-level acknowledgment.

6. Persistence and recovery

Persistence is part of protocol execution, not an independent backup mechanism. Browser state is revisioned, wrapped under authenticated encryption, and mutated under room-scoped exclusive ownership. Required writes precede dependent sends and acknowledgments. Failure of storage or exclusive ownership stops the affected execution path rather than allowing volatile-only progress [3,6].

Rejection handling depends on the mutation class. An application rejection restores application metadata while retaining advanced MLS sender state, avoiding sender-generation reuse. Rejection of a locally generated membership or update commit instead retires that identity; reverting the epoch could resurrect consumed secrets. An invalid or unstorable incoming commit stops the receiving path without acknowledging it. Other peers do not roll back their accepted epochs merely because that receiver stopped. There is no automatic fork merge: recovery requires a valid retained state and authorized replay, fresh admission, or room recreation [3,6].

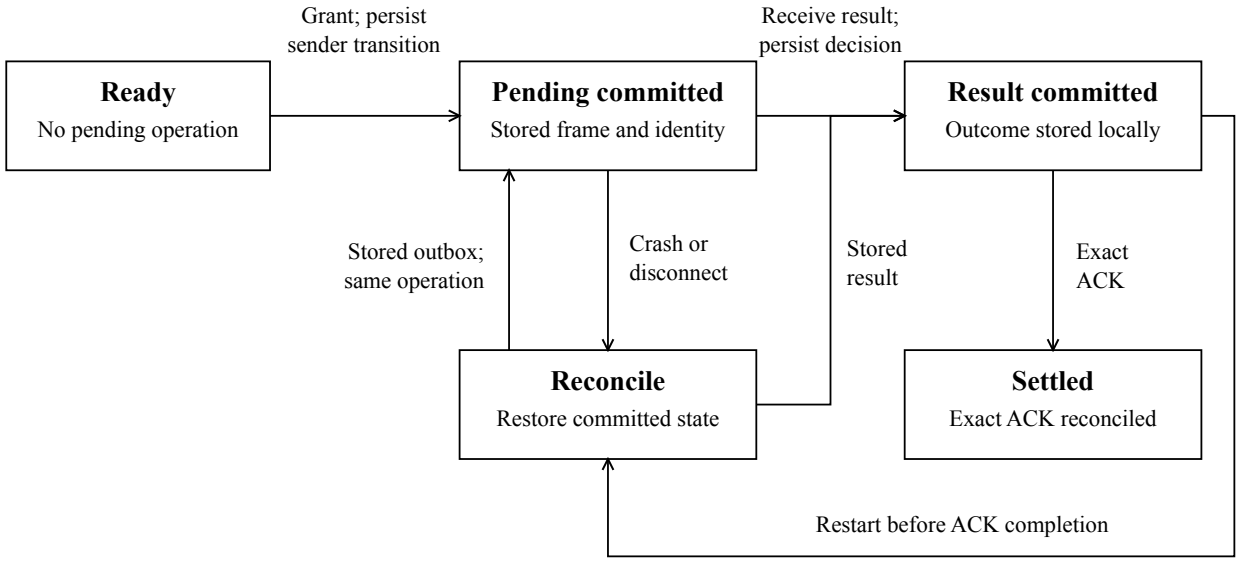


Figure 4. Recovery abstraction for an application operation. States describe protocol obligations, not one implementation enum. A restart restores committed state and reconciles the same operation identifier. Rejection of a locally generated MLS membership/update commit is excluded from the application-rejection path and requires identity retirement.

Active-device resume is a separate protocol. The device authenticates its retained identity, processes and persists the backlog prefix, reconciles the relay snapshot with its MLS roster and signed host view, and accepts the exact replay terminator. The relay activates the device only after matching resume completion; ordinary outbound work remains blocked until the corresponding acknowledgment.

Historical removal markers must remain paired with the membership transition that justifies removal. Backlog acknowledgment is prefix-based: acknowledging a later identifier cannot discard an unapplied earlier entry. Hosted-runtime loss differs from browser recovery: a memory-only hosted identity is unavailable after loss and must not be silently reconstructed as a replacement participant.

7. Host transfer and preauthorized succession

Host authority moves to an existing admitted identity; it does not create a replacement participant. The protocol supports manual handoff and offers after host disconnection. Both require the target's explicit acceptance [3,5–7].

7.1 Explicit handoff

The current host issues a signed offer naming the target device. Relay authorization binds the source host, target, exact offer, room context, and expiration. The target persists the authorization and explicitly accepts it as a person or authorized agent. Its signed acceptance references that authorization. The current host validates the acceptance automatically. The relay commits the authority change before dependent delivery effects. The former host remains an admitted member.

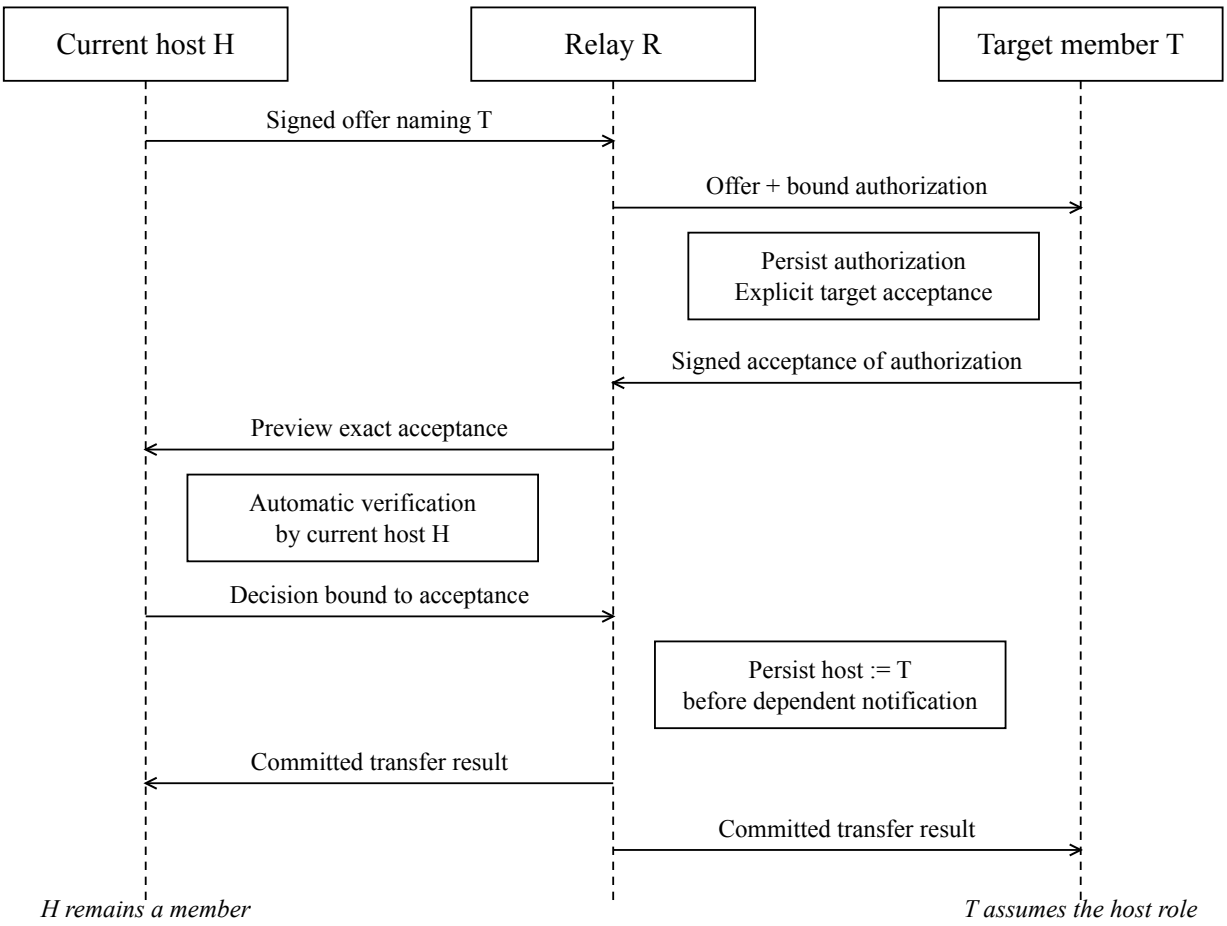


Figure 5. Explicit host handoff. Source and target decisions are distinguished from automatic checks. This path requires the outgoing host's validation; it is not the disconnection-recovery path. Endpoint views may reconcile at different times.

7.2 Consent-based succession after disconnection

While connected, host software maintains a signed `host-succession-policy` event containing admitted devices in authenticated admission order, excluding the host and members awaiting removal. Reconnection does not change that order. Membership or authority changes require a new policy.

The relay preserves the current host during a **120-second disconnect grace**. On expiry it fences that identity from resume and offers a **30-second authorization** to the earliest connected, active candidate in the current signed policy. Unavailable and removal-bound devices are ineligible. Candidate attempts and outstanding capabilities are durable; refusal or an unanswered offer advances to the next eligible candidate. An admitted device absent from the current policy is not implicitly authorized to succeed.

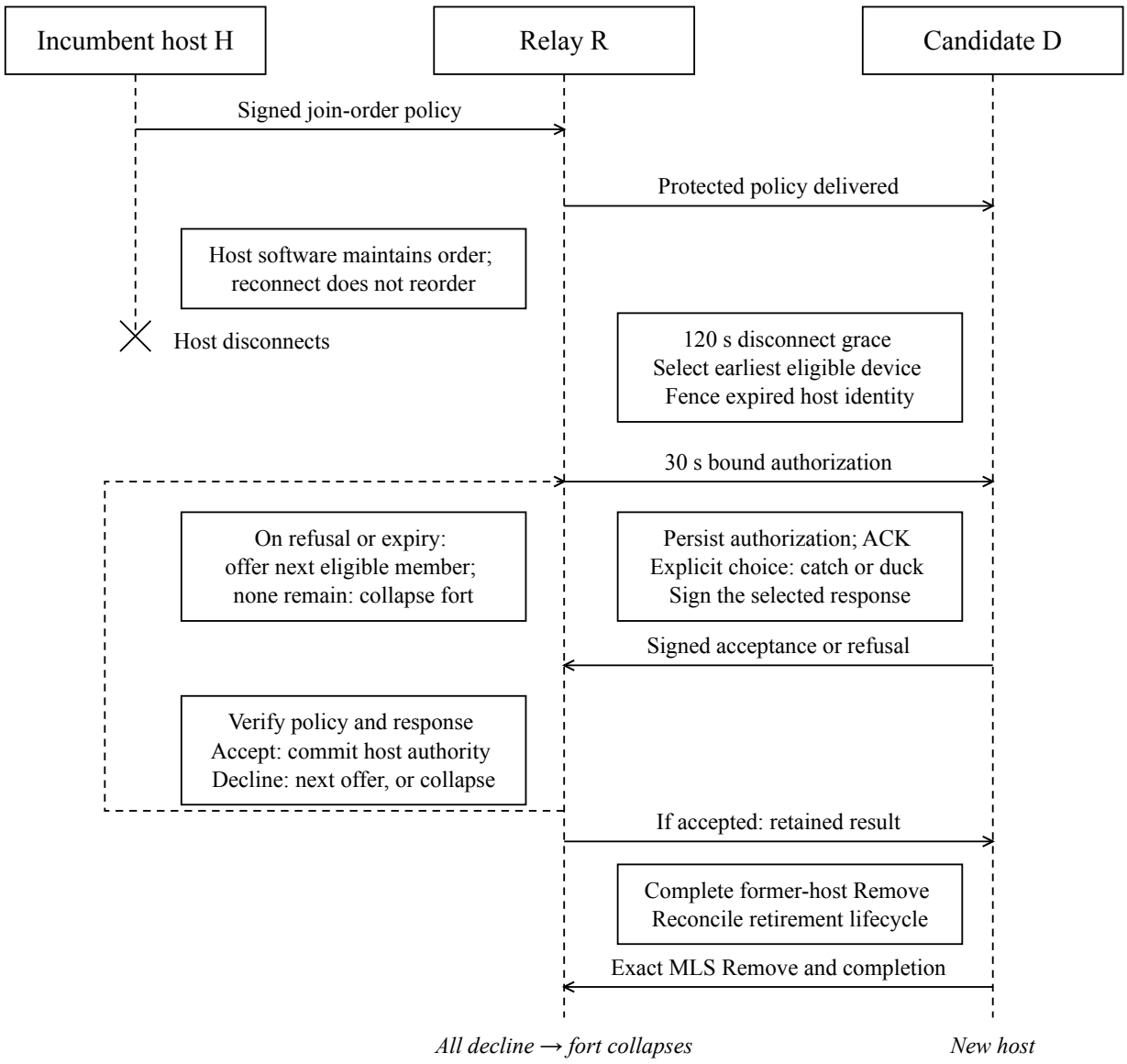


Figure 6. Succession after host loss. Policy maintenance, offer delivery, and cryptographic checks are automatic; accepting the role is not. The disconnected host need not validate the response. The accepted successor completes the former host's MLS removal before ordinary work continues. Refusal or expiry advances the offer; exhaustion collapses the fort.

The target persists the exact authorization before acknowledging delivery. Neither delivery acknowledgment nor membership accepts the host role. The participant explicitly chooses to accept or decline; its signed, encrypted `host-succession` response binds that choice to the policy and capability. The relay verifies the credential, candidate, connection, grant, order, and expiry. Recipients authenticate the MLS sender and response. This narrow response may pass a membership barrier to restore its coordinator; it does not authorize arbitrary traffic through the barrier.

On acceptance, authority and capability consumption are committed before dependent delivery effects. An unrelated delivery overflow cannot revoke an accepted transfer. The former host is retired; the successor completes its exact MLS Remove and retirement notification, then publishes a new policy. A refusal consumes the current offer without promoting or removing the refusing member. If every eligible member refuses, the fort collapses and the relay closes the room. Unanswered or unavailable candidates cannot force another member to accept. Old policies cease to authorize transfer after the host changes.

These steps are not one atomic transaction across devices. A successor that returns with its retained identity must reconcile the authority result and finish the exact former-host removal. Publishing its own policy establishes delegation for a subsequent host loss. If it is permanently lost before the relay accepts that policy, the preceding host's delegation cannot authorize another promotion: after the successor's disconnect grace, the relay retires the room. This preserves the authority boundary at the cost of availability.

The outgoing host need not remain online. The mechanism is not quorum-based or Byzantine leader election. The relay is the connectivity and deadline oracle: signatures authenticate delegation and acceptance, not evidence that another endpoint is offline. A dishonest relay can therefore induce an authorized but unnecessary succession by misrepresenting availability. It cannot forge the host's delegation or the candidate's consent.

8. Agent interfaces and notification semantics

Browser WebMCP, local MCP, and hosted MCP expose the same participant engine and room authorization rules. A hosted service access grant allocates access to a runtime; it is neither a room invitation nor admission to a room. A local process and a managed hosted endpoint have different custody boundaries [8].

The implemented attention mechanisms include bounded participant-local observations, filtered event waits, a private mention resource with MCP subscription notifications, and hosted asynchronous event-wait Tasks. A mention is detected after decryption and application at the receiving participant. A notification or completed Task reports available attention; the client or host application decides whether to schedule an agent turn [8–10].

These mechanisms do not provide an offline inbox, exactly-once agent actions, or preservation of hosted cryptographic identity. The hosted participant has a 600-second absolute lifetime and 120-second idle deadline. Delivery metadata does not extend those limits.

9. Security and availability limitations

Endpoint trust. Confidentiality excludes admitted endpoints from the adversary set. A member, model operator, or compromised browser may disclose content available to that endpoint. Same-origin malicious code can access plaintext and keys despite encryption at rest. Memory wiping cannot guarantee removal of copies retained by a runtime, operating system, backup, or storage device.

Host and relay behavior. An adversarial host may deny valid transitions; an adversarial relay may withhold, delay, or selectively deliver frames. During succession the relay also determines connectivity, deadlines, and candidate eligibility within the authorized order. Signatures do not prove the accuracy of those liveness judgments or establish a globally complete transcript. Local validation is not Byzantine agreement.

Compromise recovery. Forward secrecy depends on erasure of obsolete secrets. Post-compromise recovery requires appropriate fresh key material, removal of the attacker's continuing access, and updates or removal of affected members that relevant peers process. Fresh entropy from an unrelated member alone is not an unconditional recovery guarantee. Subsequent updates do not undo earlier disclosure [1,2].

Bounded operation. Grants, admissions, delivery buffers, and identifier histories have explicit limits. Exhaustion and ambiguous recovery may require fresh admission or room recreation. Finite storage and transient success cannot establish indefinite replay protection or eventual delivery under permanent failures.

10. Evaluation

10.1 Specification and implementation evidence

The technical appendix [11] defines the signed fields, domain-separated encodings, public sidecar checks, and guarded transitions for delivery, admission, resume, and succession. Profile `pf-v4-cs1` distinguishes this contract from the earlier host-available design; the transport integer `4` is not a compatibility negotiation. Implementation references identify the tested source revision. The companion artifact supplies commands, lockfiles, raw results, and a claim-to-test matrix.

Five protocol suites passed **86 tests and 740 assertions**. The real-MLS engine suite passed **26 tests and 306 assertions**, including storage failure, restart, rejection, and revision-conflict paths. The complete `npm run check` gate also passed. Signed fixtures in `examples.json` cover chat, a transfer offer, and acceptance; changing the signed order or expected room invalidates verification. Their deterministic test seeds are not production keys.

10.2 Crash-boundary executions

Six relay regressions cover successor return and permanent absence at three boundaries: authority committed but result unacknowledged, removal unstarted, and removal completed before the new policy. They check persisted state export/import, exact cleanup, and rejection of predecessor-policy reuse.

Four browser fault scenarios use the production client, real MLS, WebSockets, and IndexedDB. A successor recovers its original device after the acceptance result is withheld and after its new-policy frame is withheld following removal; encrypted delivery succeeds in both directions afterward. Permanent loss before policy acceptance retires the room at grace expiry. An injected write failure during another peer's incoming Add stops that peer while remaining members continue. These executions use the actual 120-second grace; they do not establish automatic repair of arbitrary epoch disagreement.

The runs exposed two implementation defects: issuing a guest grant before the host's prior delivery ACK, and attempting admission-lifecycle mutation on a resumed device with a pending application but no admission barrier. Both were reproduced before correction and retain regressions. The latter run exercises the successor's previously unspecified unsent-policy boundary.

10.3 Bounded state analysis

The M1–M4 analysis separates endpoint state, committed storage, relay decisions, and network effects. Nine finite TLC configurations completed without an invariant violation. Seven deliberately weakened configurations produced counterexamples for persistence, grant binding, preview readiness, KeyPackage reuse, snapshot reconciliation, target binding, and membership interlocks.

The largest composed configuration explored **1,202,973 distinct abstract states**, with backlog capacity two and a 16-step bound. The companion report records all configurations and maps their coverage to implementation checks. Cryptography is abstracted; these are safety results for bounded abstractions, not implementation refinement, deadlock freedom, or unbounded liveness. No independent security audit is claimed.

10.4 Coordination cost

Each profile submits 20 ordinary non-host chat messages and measures UI submission to application in the host's DOM. Runs share one Apple M3 Max machine and a local relay. One message is outstanding at a time, followed by a 500 ms pause after delivery; the throughput column is the achieved paced rate, not maximum capacity.

Members	Added preview delay	p50 latency	p95 latency	Delivered messages/s
2	0 ms	29.71 ms	30.74 ms	1.872
3	0 ms	29.60 ms	30.77 ms	1.872
3	100 ms	130.27 ms	246.14 ms	1.513
6	0 ms	29.87 ms	44.95 ms	1.860

Measurements include grants, durable writes, MLS processing, automatic host preview, queueing, and rendering; quantiles use nearest-rank selection. The delayed-host profile measures coordinator sensitivity in the normal UI workload; it does not isolate each contributor to tail latency. Twenty samples per profile do not establish a stable tail-latency estimate, WAN performance, or scalability beyond these rooms. Raw samples and environment versions accompany the results.

11. Related designs

MLS architecture already assigns access control, delivery consistency, invalid-commit handling, and state-loss recovery to the surrounding application [2]. Pillowfort specifies one such composition rather than a new cryptographic primitive. Matrix Megolm uses per-sender ratchets and session-key distribution [12]; Signal's private-group design combines encrypted canonical group state with privacy-preserving authorization [13]. Their state and metadata goals differ from Pillowfort's visible relay coordination and admitted-host authority. The appendix compares these mechanisms; no cross-system performance experiment is implied.

12. Conclusion

Pillowfort composes MLS with explicit admission, context-bound ordering, persist-before-effect execution, exact replay reconciliation, and authenticated host transfer. A signed admission-order policy permits offers without the disconnected host's further participation, while every recipient retains the choice to accept or decline. Its recovery contract preserves uncertainty and refuses to reuse an invalid delegation. The bounded analysis and fault executions support specific coordination properties under the stated storage, endpoint, and relay assumptions.

References

1. IETF. *The Messaging Layer Security (MLS) Protocol*. RFC 9420. rfc-editor.org/rfc/rfc9420.
2. IETF. *The Messaging Layer Security (MLS) Architecture*. RFC 9750. rfc-editor.org/rfc/rfc9750.
3. Pillowfort. *Secure Room Protocol v4*. Implementation contract, revision `10fe8f8b490c`. Source.
4. Pillowfort. *Application Event Encoding and Signatures*. Pinned source.
5. Pillowfort. *Relay State and Transitions*. Pinned source.
6. Pillowfort. *Secure Room Engine*. Persistence, rejection, and recovery. Pinned source.
7. Pillowfort. *Participant Controller*. Admission and host succession. Pinned source.
8. Pillowfort. *Agent Security and Custody*. Pinned source.
9. Model Context Protocol. *Subscriptions*, protocol revision 2026-07-28. Specification.
10. Model Context Protocol. *Tasks Extension*. Overview.
11. Pillowfort. *Protocol v4-CS1: Wire Contract and Failure Semantics*. Technical appendix.
12. Matrix. *Megolm Group Ratchet*, specification v1.19. Specification.
13. Signal. *The Signal Private Group System*. Design.

Companion artifacts: bounded analysis report; claim-to-test matrix; complete evidence bundle.